МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

Кафедра оптоелектроніки та інформаційних технологій

Звіт

До лабораторної роботи №2

Варіант 13

На тему “ Функції вищого порядку, lambda, композиція”

Виконала:

Студентка групи ФЕП-21

Сокирка А. І.

Перевірив:

Ас. Остріков Д. А.

Львів 2026

Мета роботи:

- ознайомитися з функціями вищого порядку та анонімними функціями lambda;
- навчитися використовувати map, filter і reduce для обробки колекцій;
- навчитися будувати композицію функцій та функціональний конвеєр перетворень;
- реалізувати обробку списку записів без мутації початкових даних;
- закріпити використання TypedDict, pytest, black, ruff і mypy

Теоретичні відомості:

Функції вищого порядку — це функції, які можуть приймати інші функції як аргументи або повертати функції як результат. Такий підхід дає змогу створювати гнучкіші та повторно використовувані операції.

Функція lambda — це короткий спосіб оголошення анонімної функції. Вона зручна для невеликих операцій, наприклад для визначення умови фільтрації або ключа сортування.

map застосовує задану функцію до кожного елемента послідовності, filter залишає лише елементи, які задовольняють певну умову, а reduce послідовно об'єднує елементи в один результат за допомогою акумулятора.

Композиція функцій — це об'єднання кількох функцій у єдиний конвеєр, де результат однієї операції передається наступній. Для уникнення мутацій під час перетворення записів можна створювати нові словники за допомогою розпакування **record.

Хід роботи

Під час виконання лабораторної роботи було розроблено програму для аналізу готельних бронювань відповідно до індивідуального варіанта 11. Кожен запис містить ідентифікатор бронювання (id), ім'я гостя (guest), місто (city) та список вартостей проживання за ночі (nights_cost).

У модулі core.py описано типізовану структуру Booking за допомогою TypedDict та реалізовано чисті функції для нормалізації імен гостей, обчислення загальної вартості бронювання, застосування коефіцієнта для вибраного міста, фільтрації за мінімальною сумою, сортування та формування Top-N.

Для побудови конвеєра використано композицію функцій compose. Перетворення створюють нові записи, не змінюючи початковий список бронювань. Для

обробки колекцій застосовано map і filter, а для підрахунку статистичних показників — reduce.

Окремо реалізовано агрегацію, яка обчислює кількість бронювань (count), загальний дохід (revenue) та середню вартість бронювання (avg_booking). У модулі app.py розміщено демонстраційні дані та виведення результатів, а main.py використовується як точка входу до програми.

Коректність роботи перевіряється автоматизованими тестами pytest, які охоплюють формування Top-N, відсутність мутацій, правильність агрегації та роботу композиції функцій.

Файл core.py

У модулі core.py реалізовано тип Booking, функції перетворення записів, композицію функцій, побудову конвеєра та reduce-агрегацію.

```python
from collections.abc import Callable, Iterable, Iterator
from functools import reduce
from itertools import islice
from typing import Any, TypedDict, cast


class Parcel(TypedDict):  9 usages
    id: int
    sender: str
    region: str
    delivery_parts: list[float | int]


class ProcessedParcel(Parcel):  16 usages
    delivery_cost: float | int


class DeliveryStats(TypedDict):  1 usage
    count: int
    delivery_sum: float | int
    avg_delivery: float | int


Pipeline = Callable[[Iterable[Parcel]], list[ProcessedParcel]]


def compose(*functions: Callable[[Any], Any]) -> Callable[[Any], Any]:  2 usages
    def composed(value: Any) -> Any:
```

```
        for function in reversed [(Any) -> Any] (functions):
            value = function(value)
        return value

    return composed


def normalize_senders(  1 usage
    parcels: Iterable[Parcel],
) -> Iterator[Parcel]:
    return map [Parcel] (
        lambda parcel :{__getitem__} : cast(
            Parcel,
            {**parcel, "sender": parcel["sender"].strip().title()},
        ),
        parcels,
    )


def with_delivery_cost(  1 usage
    parcels: Iterable[Parcel],
) -> Iterator[ProcessedParcel]:
    return map [ProcessedParcel] (
        lambda parcel :{__getitem__} : cast(
            ProcessedParcel,
            {
                **parcel,
                "delivery_cost": float(sum(parcel["delivery_parts"])),
            },
        ),
        parcels,
    )


def apply_region_coefficient(  1 usage
    region: str,
    factor: float | int ,
) -> Callable[[Iterable[ProcessedParcel]], Iterator[ProcessedParcel]]:
    def transform(
        parcels: Iterable[ProcessedParcel],
    ) -> Iterator[ProcessedParcel]:
        return map [ProcessedParcel] (
            lambda parcel :{__getitem__} : cast(
                ProcessedParcel,
                {
                    **parcel,
                    "delivery_cost": (
                        parcel["delivery_cost"] * factor
                        if parcel["region"] == region
                        else parcel["delivery_cost"]
                    ),
```

```
                    ),
                },
            ),
            parcels,
        )

    return transform


def only_above_threshold(  1 usage
    threshold: float | int,
) -> Callable[[Iterable[ProcessedParcel]], Iterator[ProcessedParcel]]:
    return lambda parcels: Iterable[ProcessedParcel]: filter(
        lambda parcel: {__getitem__}: parcel["delivery_cost"] >= threshold,
        parcels,
    )


def sort_by_delivery_desc(  1 usage
    parcels: Iterable[ProcessedParcel],
) -> list[ProcessedParcel]:
    return sorted(
        parcels,
        key=lambda parcel: ProcessedParcel: (-parcel["delivery_cost"], parcel["id"]),
    )
```

```
def take(  1 usage
    count: int,
) -> Callable[[Iterable[ProcessedParcel]], list[ProcessedParcel]]:
    return lambda parcels: Iterable[ProcessedParcel]: list[_T](islice[ProcessedParcel](parcels, max(0, count)))


def reduce_stats(  2 usages
    parcels: Iterable[ProcessedParcel],
) -> DeliveryStats:
    count, delivery_sum = reduce(
        lambda acc: {__getitem__}, parcel: {__getitem__}: (
            acc[0] + 1,
            acc[1] + parcel["delivery_cost"],
        ),
        parcels,
        (0, 0.0),
    )

    average = delivery_sum / count if count else 0.0

    return {
        "count": count,
        "delivery_sum": delivery_sum,
        "avg_delivery": average,
    }


def build_pipeline(  5 usages
```

```python
def build_pipeline(  5 usages
    top_n: int = 3,
    region: str = "West",
    factor: float | int = 1.1,
    threshold: float | int = 75.0,
) -> Pipeline:
    pipeline = compose(
        *functions: take(top_n),
        sort_by_delivery_desc,
        only_above_threshold(threshold),
        apply_region_coefficient(region, factor),
        with_delivery_cost,
        normalize_senders,
    )

    return cast(Pipeline, pipeline)
```

Файл app.py

У модулі app.py створено приклади готельних бронювань, виконано запуск конвеєра та виведено список найкращих бронювань і статистичні показники.

```python
import argparse
import json
from pathlib import Path
from typing import cast

from core import (
    Parcel,
    ProcessedParcel,
    build_pipeline,
    reduce_stats,
)


def load_parcels() -> list[Parcel]:  1 usage
    data_path = Path(__file__).with_name("sample_data.json")

    with data_path.open( mode: "r", encoding="utf-8") as file:
        return cast(list[Parcel], json.load(file))


def run() -> None:  2 usages
    parser = argparse.ArgumentParser(
        description="Parcel delivery cost analysis"
    )
    parser.add_argument( *name_or_flags: "--top", type=int, default=3)
    parser.add_argument( *name_or_flags: "--boost-region", default="West")
    parser.add_argument( *name_or_flags: "--factor", type=float, default=1.1)
    parser.add_argument( *name_or_flags: "--threshold", type=float, default=75.0)
```

```
    args = parser.parse_args()

    parcels = load_parcels()

    pipeline = build_pipeline(
        top_n=args.top,
        region=args.boost_region,
        factor=args.factor,
        threshold=args.threshold,
    )

    result: list[ProcessedParcel] = pipeline(parcels)
    stats = reduce_stats(result)

    print("Parcel delivery report")
    print("-" * 35)

    for parcel in result:
        print(
            f"Parcel {parcel['id']}: "
            f"{parcel['sender']} | "
            f"{parcel['region']} | "
            f"{parcel['delivery_cost']:.2f} UAH"
        )

    print("-" * 35)
```

```
    print("-" * 35)
    print(f"Number of parcels: {stats['count']}")
    print(f"Total delivery cost: {stats['delivery_sum']:.2f} UAH")
    print(f"Average delivery cost: {stats['avg_delivery']:.2f} UAH")


if __name__ == "__main__":
    run()
```

Файл test_core.py

У файлі test_core.py створено автоматизовані тести для перевірки правильності Top-N, незмінності вхідних даних, обчислення статистики та коректної роботи композиції.

```python
from copy import deepcopy

from core import (
    Parcel,
    ProcessedParcel,
    build_pipeline,
    compose,
    reduce_stats,
)


def sample_parcels() -> list[Parcel]:  4 usages
    return [
        {
            "id": 1,
            "sender": "anna kovalenko",
            "region": "West",
            "delivery_parts": [50, 30, 20],
        },
        {
            "id": 2,
            "sender": "mark petrenko",
            "region": "East",
            "delivery_parts": [40, 30],
        },
        {
            "id": 3,
            "sender": "oleg shevchenko",
```

```
            "region": "West",
            "delivery_parts": [80, 50],
        },
        {
            "id": 4,
            "sender": "sofia melnyk",
            "region": "North",
            "delivery_parts": [60, 20],
        },
        {
            "id": 5,
            "sender": "ivan bondar",
            "region": "West",
            "delivery_parts": [30, 20],
        },
        {
            "id": 6,
            "sender": "daria kravets",
            "region": "East",
            "delivery_parts": [100, 50],
        },
    ]


def test_top_n():
    result = build_pipeline(top_n=3)(sample_parcels())
```

```
    assert [parcel["id"] for parcel in result] == [6, 3, 1]


def test_no_mutation():
    parcels = sample_parcels()
    original = deepcopy(parcels)

    build_pipeline()(parcels)

    assert parcels == original


def test_reduce_stats():
    result = build_pipeline()(sample_parcels())
    stats = reduce_stats(result)

    assert stats["count"] == 3
    assert stats["delivery_sum"] == 403.0
    assert abs(stats["avg_delivery"] - 134.3333333333) < 0.0001


def test_compose():
    add_one = lambda value :{__add__} : value + 1
    double = lambda value :{__mul__} : value * 2

    combined = compose( *functions: add_one, double)
```

```
    assert combined(3) == 7


def test_region_coefficient_and_threshold():
    result = build_pipeline(
        top_n=10,
        region="West",
        factor=1.1,
        threshold=100,
    )(sample_parcels())

    assert all(parcel["delivery_cost"] >= 100 for parcel in result)
    assert result[0]["delivery_cost"] == 143.0
    assert result[1]["delivery_cost"] == 110.00000000000001
```

Файл main.py

Файл main.py є точкою входу до програми. Він імпортує функцію run з модуля app.py та викликає її під час безпосереднього запуску файлу.

```
"""Entry point for Laboratory Work 2, variant 13."""

from app import run


def main() -> None:  1 usage
    """Run the parcel delivery analysis program."""
    run()


if __name__ == "__main__":
    main()
```

Запуск програми

Для перевірки роботи програми її було запущено з командного рядка. Після запуску відображаються Top-*N* бронювань, загальна кількість записів, загальний дохід та середня вартість бронювання.

```
C:\msys64\ucrt64\bin\python.exe "C:\Users\4myryk\Desktop\Лаби сем 3\ФП\lab02\main.py"
Parcel delivery report
----------------------------------
Parcel 6: Daria Kravets | East | 150.00 UAH
Parcel 3: Oleg Shevchenko | West | 143.00 UAH
Parcel 1: Anna Kovalenko | West | 110.00 UAH
----------------------------------
Number of parcels: 3
Total delivery cost: 403.00 UAH
Average delivery cost: 134.33 UAH

Process finished with exit code 0
```

Висновок:

Під час виконання лабораторної роботи було реалізовано функціональний конвеєр для обробки готельних бронювань. На практиці було закріплено використання функцій вищого порядку, lambda, map, filter і reduce, а також побудову композиції функцій. Було створено типізовану структуру запису Booking та реалізовано чисті трансформації для обчислення вартості бронювання, застосування коефіцієнта для вибраного міста, фільтрації, сортування й формування Top-N. Окремо реалізовано reduce-агрегацію для визначення кількості бронювань, загального доходу та середньої вартості. Під час обробки початкові записи не змінюються, оскільки для результатів перетворень створюються нові словники. Логіку обчислень відокремлено від введення та виведення даних. Автоматизовані тести дають змогу перевірити правильність роботи основних частин програми.